

Project Proposal: Characterizing the Efficiency Horizon for Memory Amortization in Frontier LLMs

University of Central Florida

February 17, 2026

Abstract

Current approaches to dynamic memory management in Large Language Models (LLMs), such as vAttention, utilize custom drivers to enable fine-grained (64KB) physical page allocation, mitigating internal fragmentation. We propose that for Retrieval-Augmented Generation (RAG) and agentic workflows characterized by massive context windows, this fragmentation becomes statistically negligible because it is a fixed cost as sequence length increases. This proposal seeks to characterize the “Efficiency Horizon” where the throughput benefits of hardware-native 2MB HugePages on stock drivers outweigh the marginal capacity loss, particularly as model depth, batch size, and tensor parallelism degrees scale.

1 Problem Statement

Internal fragmentation in KV cache management is a significant bottleneck for batch size and throughput.[1] To address this, state-of-the-art solutions like vAttention decouple virtual and physical memory, often employing custom drivers to reduce page size from the standard 2MB to 64KB.[1]

However, we hypothesize that this optimization is primarily beneficial for short-context or mixed-length workloads. In long-context regimes, the fixed overhead of fragmentation—determined by the number of model layers, batch size, and the degree of tensor parallelism is effectively amortized by the linear growth of the utilized memory. We aim to prove that for context-heavy applications, standard 2MB pages provide superior system performance by leveraging hardware-native memory management without the complexity of custom kernel drivers.

2 Mathematical Formulation

2.1 System-Wide Fragmentation Waste

In the vAttention architecture, virtual memory contiguity is maintained on a per-layer basis to support unmodified attention kernels.[1] The framework reserves $2L$ virtual buffers for each worker. Consequently, internal fragmentation is isolated to the “tail” physical page of each buffer. If we assume that, on average, half of the tail page is wasted, we can define the **Average System-Wide Fragmentation Waste** (W_{avg}):

$$W_{avg} = 2BkL \frac{P_{size}}{2} = BkLP_{size} \quad (1)$$

Where:

- L : Number of Transformer Layers (e.g., 94 for Qwen3-235B).

- P_{Size} : Page Size (2MB for stock NVIDIA drivers).
- B : Batch Size (number of concurrent requests).
- k : Tensor Parallelism (TP) degree (number of workers across which heads are split).

2.2 Impact of Parallelism Strategies

A critical novelty of this characterization is the distinction between Tensor Parallelism (TP) and Pipeline Parallelism (PP) regarding memory waste:

- **Tensor Parallelism (k):** TP acts as a **fragmentation multiplier**. Because each TP worker manages a slice of the attention heads but replicates the full $2 \times L$ buffer structure for its portion, the total number of system-wide fragmentation points increases linearly with k .
- **Pipeline Parallelism (p):** PP is **fragmentation neutral**. PP partitions the L layers across p workers, meaning each worker only manages L/p layers. The total number of system-wide buffers remains $2 \times L$ per request, maintaining a constant fragmentation footprint regardless of the PP degree.

2.3 The Amortization Curve

The **Percentage Waste** ($W_{\%}$) is the ratio of fixed waste to total allocated memory. As context length C increases, the fragmentation percentage asymptotically approaches zero:

$$W_{\%}(C) = \frac{W_{avg}}{(CS_{token}) + W_{avg}} \quad (2)$$

Table 1: Amortization of Fragmentation for Qwen3-235B ($L = 94, k = 1$) using 2MB Pages

Context Length (Tokens)	Utilized Memory (M_{util})	Fixed Waste (W_{avg})	Total Allocated (M_{total})	% Waste ($W_{\%}$)
1,000	94 MB	188 MB	282 MB	66.7%
32,000	3,008 MB	188 MB	3,196 MB	5.9%
64,000	6,016 MB	188 MB	6,204 MB	3.0%
128,000	12,032 MB	188 MB	12,220 MB	1.5%

Table 2: Amortization of Fragmentation for Qwen3-235B ($L = 94, k = 2$) using 2MB Pages

Context Length (Tokens)	Utilized Memory (M_{util})	Fixed Waste (W_{avg})	Total Allocated (M_{total})	% Waste ($W_{\%}$)
1,000	94 MB	376 MB	470 MB	80.0%
32,000	3,008 MB	376 MB	3,384 MB	11.1%
64,000	6,016 MB	376 MB	6,392 MB	5.9%
128,000	12,032 MB	376 MB	12,408 MB	3.0%
256,000	24,064 MB	376 MB	24,440 MB	1.5%

2.4 Memory Footprint per Token (S_{token})

The memory required to store the KV cache for a single token across all layers is derived from model architectural parameters:

$$S_{token} = 2LH D_{head} P_{byte} \quad (3)$$

Where H is the number of attention heads, D_{head} is the head dimension, and P_{byte} is the precision. For Qwen3-235B at 8-bit precision ($P = 1$), $S_{token} \approx 94$ KB.

3 Proposed Research

3.1 Hypothesis

We hypothesize that for any given model depth (L) and parallelism degree (k), there exists an threshold where context size C renders the 2MB page fragmentation practically insignificant ($< 2\%$).

3.2 Methodology

- **Characterization:** We will test the fragmentation curve using a cluster to validate the compounding relationship between L, B, k , and $W_{\%}$.
- **Validation:** We will compare the throughput of vAttention on stock drivers against vLLM to demonstrate that hardware-native 2MB pages yield superior performance once fragmentation is amortized.
- **Novelty:** We fill the gap in prior work by characterizing fragmentation as a dynamic function of workload scale and parallelism strategy rather than just allocation granularity.

References

- [1] Ramya Prabhu, Ajay Nayak, Jayashree Mohan, Ramchandran Ramjee, and Ashish Panwar. vattention: Dynamic memory management for serving llms without pagedattention. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (ASPLOS '25)*, Rotterdam, Netherlands, 2025. ACM.